Hidro Smart

Informe de diseño de software

Emerson Calet Molano Cardozo

José Alejandro Montealegre Rodríguez

José David Amaya Mosquera

Instructor

José de Jesús Motta Vargas

Servicio Nacional de Aprendizaje - SENA

Análisis y Desarrollo de Software - ADSO

Ficha - 3145555

2026

Resumen

El presente Informe de Diseño de Software describe la arquitectura, módulos, base de datos, diagramas UML y especificaciones técnicas del sistema Hidro Smart, una solución IoT para el monitoreo inteligente del consumo de agua en hogares. El sistema permite visualización en tiempo real, alertas automáticas, recomendaciones de ahorro, reportes exportables, modo vacaciones, metas de consumo, historial de conectividad de dispositivos y soporte técnico integrado. Se utiliza React + Vite para el frontend (web/móvil responsivo), Node.js para el backend, PostgreSQL como base de datos, y se aplican patrones de arquitectura moderna como Hexagonal, Event-Driven, Circuit Breaker y DTOs. El diseño se basa en el SRS v3.0 actualizado.

Palabras clave: Consumo de agua, IoT, React, Vite, Node.js, sostenibilidad, alertas inteligentes, arquitectura hexagonal, event-driven.

Tabla de Contenido

Introducción 4

Descripción general del sistema 7

Arquitectura del Software 9

Tipo de Arquitectura 10

Arquitectura en Capas 10

Flujo general de datos 11

Diseño de Componentes o Módulos 11

Diagramas del Sistema (UML) 13

Diagrama de Casos de Uso 14

Diagrama de Clases (Diagrama de Dominio) 14

Diagrama de Actividad 14

Diagrama de Transición de Estados 14

Diagrama de Secuencia 15

Diagrama de Componentes 15

Diagrama de Despliegue 15

Diseño de Base de Datos 16

Modelo Entidad - Relación 16

Tablas principales 16

Detalles de tablas principales (campos, llaves primarias y foráneas) 17

Relaciones entre tablas 18

Diseño de Interfaces 20

Prototipos de pantallas principales 20

Formularios principales 21

Menús y navegación 21

Reglas de negocio 22

Seguridad del Sistema 24

Autenticación 24

Autorización 24

Control de acceso 24

Protección de datos 25

Manejo de contraseñas 25

Especificaciones Técnicas 26

Concideraciones de Implementación 28

Estándares de codificación 28

Estructura del proyecto 28

Manejo de errores 29

Pruebas del sistema 29

Otras consideraciones 29

Referencias 31

Control de actualización

Fecha de corte: 2026-10-09. Estado: Existente pero actualizado.

Fuente principal de esta actualización: Hidro Smart Informe de Diseño de Software v1.2.docx y documentación técnica de PT_HS.

Se conservan arquitectura, interfaces, patrones, tablas, estructura de módulos y diagramas del diseño original. Se ajustan las afirmaciones que no coinciden con el stack y el corte de verificación actual; el anexo resume la línea base vigente.

Introducción

El presente documento constituye el Informe de Diseño de Software del proyecto Hidro Smart, desarrollado como parte del Tecnólogo en Análisis y Desarrollo de Software del Servicio Nacional de Aprendizaje (SENA), ficha 3145555.

Propósito del documento: Este documento tiene como propósito principal describir de forma detallada y técnica cómo se va a construir el sistema Hidro Smart, tomando como base la Especificación de Requisitos de Software (SRS v1.1) actualizada. Sirve como guía oficial para el equipo de desarrollo durante las fases de:

- Construcción de la base de datos (PostgreSQL).
- Desarrollo del frontend (React + Vite).
- Implementación del backend (Node.js + Express).
- Codificación de la lógica de negocio (Arquitectura Hexagonal, Casos de Uso).
- Integración de dispositivos IoT (MQTT).

- Realización de pruebas funcionales y no funcionales.

Así mismo, garantiza la trazabilidad entre los requerimientos definidos y su implementación técnica, documentando los patrones arquitectónicos y de diseño aplicados para garantizar escalabilidad, mantenibilidad y tolerancia a fallos.

Alcance del sistema

Hidro Smart es una solución tecnológica compuesta por un medidor inteligente de agua (basado en ESP32) conectado mediante IoT (MQTT local sin TLS; TLS/ACL de producción pendiente) y una aplicación web/móvil responsiva desarrollada con React + Vite. El sistema permite a los usuarios domésticos:

- Monitorear el consumo de agua en tiempo real con polling HTTP; WebSockets no verificado.
- Clasificar el consumo por actividades (ducha, lavado, aseo, cocina, etc.).
- Recibir alertas automáticas por consumo excesivo, fugas o desconexión de dispositivos.
- Generar recomendaciones personalizadas de ahorro.
- Configurar topes de consumo, idioma, moneda y temas visuales.
- Exportar reportes en PDF y Excel.
- Gestionar hogares (proyectos) y dispositivos IoT.
- Activar modo vacaciones para suspender notificaciones no críticas.
- Establecer metas de ahorro y presupuesto mensual.
- Solicitar soporte técnico a través de un sistema de tickets.

- Consultar historial de conectividad de dispositivos (señal, batería, voltaje, temperatura).
- Eliminar su cuenta de forma autónoma (derecho al olvido).

Quedan fuera del alcance inicial: integración con sistemas externos de acueductos públicos y la versión nativa para iOS (se prioriza web responsiva).

Definiciones y términos

Término	Definición
IoT	Internet of Things: red de dispositivos físicos (medidores de agua) conectados a internet que envían datos de consumo en tiempo real.
ESP32	Microcontrolador con Wi-Fi y Bluetooth usado como medidor inteligente.
Hogar/Proyecto	Entidad que agrupa usuarios, dispositivos y configuraciones de consumo dentro del sistema.
Estrato socioeconómico	Clasificación usada para establecer valores base de referencia de consumo (1 a 6).
Token Access/Refresh	Mecanismos de autenticación JWT para gestionar sesiones seguras.
Lectura	Registro individual de caudal y consumo enviado por el dispositivo IoT.

Término	Definición
Alerta	Notificación automática generada cuando se supera un tope configurado o se detecta una anomalía.
DTO	Data Transfer Object: objeto que estructura datos entre capas.
Event Bus	Sistema de comunicación asíncrona basada en eventos (publicador/suscriptor).
Circuit Breaker	Patrón de tolerancia a fallos que evita llamadas a servicios caídos.
Vite	Bundler ultrarrápido para aplicaciones web modernas.
React	Biblioteca de JavaScript para construir interfaces de usuario basadas en componentes.

Referencias de documentos relacionados

- Amaya Mosquera, J. D., Molano Cardozo, E. C. & Montealegre Rodríguez, J. A. (2025). Diseño conceptual y arquitectura de información. SENA. https://drive.google.com/drive/folders/1O1brvYi-oYlElVLB6hLb7HU881uh8dJU?usp=sharing
- Motta Vargas, J. de J. (2025). Acta General del Proceso del Proyecto Hidro Smart. SENA. https://drive.google.com/drive/folders/1Ts6xxuiW3DcT0RjO286Ns21UE2N0LR2N?usp=drive_link

- Amaya Mosquera, J. D. et al. (2026). Encuesta sobre el Consumo de Agua – Hidro Smart [Formulario Google]. https://forms.gle/er5mLrWgxdPGptHL9
- Equipo Hidro Smart. (2026). Mapa de Procesos. https://drive.google.com/drive/folders/1l84t0bhXedeTjBIChnlTzT97f-v6jG-g?usp=drive_link
- Equipo Hidro Smart. (2026). Modelo Entidad-Relación (MER). https://drive.google.com/drive/folders/10q-U9_k-FVHzPtbMjG2GP4O6VEaU28QJ?usp=drive_link
- Equipo Hidro Smart. (2026). Wireframes de Interfaz – Figma [Prototipo web y móvil]. https://drive.google.com/drive/folders/1W8b38qeR7oNWkIHniYXydKEc5ZTbtNnn?usp=drive_link

Descripción general del sistema

Hidro Smart es un sistema inteligente diseñado para resolver el problema de la falta de visibilidad y control del consumo de agua en los hogares colombianos. Combina un medidor físico inteligente (IoT) con una aplicación web/móvil que procesa, analiza y presenta la información en tiempo real..

Diseñar e implementar una solución tecnológica que permita a los usuarios domésticos monitorear, controlar y optimizar su consumo diario de agua, proporcionando información detallada por actividad, alertas preventivas, herramientas de administración avanzada (modo vacaciones, metas, soporte) y recomendaciones prácticas para reducir el desperdicio y promover el uso responsable del recurso hídrico.

El sistema contempla tres tipos de usuarios principales:

Tipo de usuario	Tipo de usuario	Rol en el sistema
Usuario doméstico	Persona que utiliza la app para consultar su consumo, configurar topes, generar reportes, gestionar metas y solicitar soporte.	Visualiza datos de sus hogares, configura topes, genera reportes, gestiona dispositivos y metas.
Administrador	Personal autorizado que supervisa hogares, usuarios y dispositivos.	Puede crear/suspender cuentas, revisar auditorías, gestionar configuraciones globales.
Soporte Técnico	Técnico IoT que realiza mantenimiento, calibraciones y soporte de conectividad.	Accede a gestión de dispositivos, historial de conectividad y tickets de soporte, pero no a datos personales de usuarios.

El sistema se organiza en los siguientes módulos principales:

Módulo	Función principal
RF1 — Autenticación y seguridad	Registro, login JWT, roles, recuperación de contraseña, actualización de perfil, suspensión de cuentas, eliminación de cuenta propia (soft delete).
RF2 — Gestión de dispositivos IoT	Registro, calibración (factor K y offset), suspensión, reactivación, desvinculación,

	umbral de alerta por dispositivo, historial de conectividad.
RF3 — Gestión de hogares	Creación de hogares, estratificación, asignación de titular y miembros, vinculación de dispositivos, suspensión y reactivación.
RF4 — Configuraciones del sistema	Topes de consumo, alertas personalizadas, recomendaciones, idioma/moneda, temas visuales, auditoría, modo vacaciones.
RF5 — Reportes y análisis	Gráficos en tiempo real (polling HTTP; WebSockets no verificado), históricos, modelo predictivo, alertas, exportación PDF/Excel, hábitos de consumo, metas de ahorro, soporte técnico (tickets).

Hidro Smart opera en un entorno doméstico conectado.

El flujo principal es el siguiente:

1. Los medidores inteligentes (ESP32) instalados en la tubería de agua envían datos de caudal y métricas de salud (señal, batería) mediante MQTT local sin TLS; TLS/ACL de producción pendiente al broker Mosquitto local.
2. El backend (Node.js) recibe las lecturas, las procesa (aplica calibración, calcula costo) y las almacena en PostgreSQL.
3. El frontend React/Vite se comunica con el backend mediante API REST; el corte actual refleja datos mediante polling HTTP y no verifica push en tiempo real.
4. El usuario interactúa desde su navegador o dispositivo móvil para ver dashboards, configurar alertas, gestionar metas, activar modo vacaciones, etc.
5. El sistema envía notificaciones push (FCM), correos electrónicos y SMS cuando se supera un tope, se detecta una fuga o se responde un ticket.

Arquitectura del software

Tipo de Arquitectura

Se adopta una Arquitectura Hexagonal (Puertos y Adaptadores) con enfoque Domain-Driven Design (DDD), combinada con Event-Driven para comunicación asíncrona y CQRS básico (resúmenes precalculados). El sistema es stateless para permitir escalado horizontal.

Características principales de la arquitectura elegida

Característica	Descripción	Beneficio para Hidro Smart
Arquitectura Hexagonal	Aísla la lógica de negocio (dominio) de los detalles externos (BD, HTTP, MQTT, etc.) mediante puertos y adaptadores.	Permite cambiar de base de datos, framework o proveedor de servicios sin afectar la lógica de negocio.
Domain-Driven Design (DDD)	Se enfoca en el dominio del problema (usuarios, hogares, dispositivos, lecturas, alertas) como núcleo del sistema.	Garantiza que las reglas de negocio estén en el centro y sean independientes de la infraestructura.
Event-Driven	Utiliza un Event Bus (mecanismo asíncrono no verificado) para	Desacopla la ingesta MQTT de las alertas y notificaciones, mejorando la resiliencia y el rendimiento.

	comunicación asíncrona entre módulos.	
CQRS básico	Separa las operaciones de escritura (transaccionales) de las de lectura (resúmenes precalculados).	Las consultas de reportes y gráficos son rápidas porque usan tablas de resumen, mientras que las escrituras mantienen la integridad transaccional.
Stateless	El backend no almacena estado en memoria local; las sesiones se gestionan con JWT + Redis no verificado (blacklist).	Permite escalar horizontalmente agregando múltiples instancias sin conflictos.

Arquitectura en Capas

Capa	Descripción	Tecnologías principales	Responsabilidad
Capa de Presentación	Interfaz de usuario web/móvil responsiva.	React 18+, Vite, Tailwind CSS	Renderizado de la UI, interacción con el usuario, validaciones en cliente.
Capa de Aplicación (Casos de Uso)	Orquestación de la lógica de negocio. Cada caso de uso es una clase independiente.	Node.js, Express	Contiene la lógica de negocio específica de cada funcionalidad, independiente de la infraestructura.
Capa de Dominio	Entidades, objetos de valor y reglas de negocio puras.	JavaScript/TypeScript (clases puras)	Representa el corazón del negocio: Usuario, Hogar, Dispositivo, Lectura, Alerta, etc.
Capa de Infraestructura	Adaptadores concretos: repositorios PostgreSQL, servicios externos (Firebase, AWS SES, Redis no verificado), cliente MQTT.	PostgreSQL, Redis no verificado, Bull, AWS SDK, Firebase Admin	Implementa los puertos definidos en la capa de aplicación y dominio (ej. IUserRepository, INotificationService).
Capa de Infraestructura	Comunicación con los medidores vía MQTT.	Mosquitto/MQTT local; cloud no verificado, MQTT.js	Recibe telemetría de los ESP32 y publica eventos en el Event Bus.
Capa de Eventos	Event Bus para comunicación asíncrona entre módulos.	EventEmitter2 (desarrollo) / procesamiento asíncrono no verificado	Desacopla la ingesta MQTT, el procesamiento de alertas y el envío de notificaciones.

		no verificado (producción)	

Flujo general de datos

El flujo de datos sigue la siguiente secuencia:

ESP32 → (MQTT local) → Broker MQTT ↓

Broker MQTT → (suscripción) → Backend (MQTT Handler) ↓

Backend → (Event Bus) → ReadingCreatedEvent ↓

ReadingCreatedHandler → (consulta/escritura) → PostgreSQL

(lectura, resúmenes) ↓

Backend → (polling HTTP; WebSockets no verificado + REST) → Frontend React

(actualización en tiempo real) ↓

Frontend React → (configuración) → Backend (API REST) ↓

Backend → (notificaciones FCM/SMTP) → Usuario (push/email).

Flujo de petición HTTP (ej. crear hogar):

HTTP Request → Middleware Chain

(Auth → Rate Limit → Validation → Audit) ↓

CreateHomeUseCase.execute(request) ↓

HomeRepository.save() (vía Unit of Work) ↓

UnitOfWork.commit() ↓

HomeResponse (DTO)

Flujo de procesamiento asíncrono (ej. generar reporte):

HTTP Request → GenerateReportUseCase ↓

Bull Queue (Redis no verificado) → Worker procesa → genera PDF/Excel↓

Guarda en S3 → Notifica al usuario vía FCM

Justificación de la arquitectura elegida

Beneficio	**Explicación**
Separación de responsabilidades	Cada capa es independiente, facilitando el mantenimiento y las pruebas.
Cada capa es independiente, facilitando el mantenimiento y las pruebas.	Stateless + colas (Bull/Redis no verificado) permiten escalar horizontalmente agregando instancias.
Tiempo real	polling HTTP; WebSockets no verificado y MQTT garantizan latencia baja para alertas y monitoreo (< 3 segundos).
Seguridad	Cifrado TLS/HTTPS en todas las comunicaciones, JWT con refresh token, RLS en PostgreSQL.
Tolerancia a fallos	Circuit Breaker + Retry Pattern evitan que fallos externos degraden el sistema.
Desacoplamiento	Event Bus desacopla la ingesta MQTT de las alertas/notificaciones, mejorando la resiliencia.
Mantenibilidad	La arquitectura hexagonal facilita cambios de tecnología (BD, servicios externos) sin tocar la lógica de negocio.

Patrones de Diseño Aplicados

Patrón	**Patrón**	**Aplicación en Hidro Smart**
Arquitectura Hexagonal (Puertos y Adaptadores)	Aísla la lógica de negocio de los detalles externos (BD,	El core (dominio + casos de uso) no depende de frameworks. Los adaptadores (repositorios, servicios externos) implementan puertos

	HTTP, MQTT, etc.).	
Event-Driven (Event Bus)	Desacopla módulos mediante eventos asíncronos.	ReadingCreatedEvent → ReadingCreatedHandler evalúa alertas. AlertTriggeredEvent → AlertTriggeredHandler envía notificaciones.
Repository Pattern	Abstrae el acceso a datos.	PostgresUserRepository, PostgresHomeRepository, etc. implementan interfaces de repositorio.
Service Layer (Casos de Uso)	Contiene la lógica de negocio pura.	Cada caso de uso es una clase con un método execute(): RegisterUserUseCase, CreateHomeUseCase.
DTO (Data Transfer Object)	Estructura las respuestas de la API.	UserResponse, HomeResponse, DeviceResponse ocultan detalles internos de las entidades.
Unit of Work (UoW)	Agrupa operaciones en una transacción ACID.	PostgresUnitOfWork maneja begin(), commit(), rollback() para operaciones que afectan múltiples tablas.
Circuit Breaker	Tolerancia a fallos en servicios externos.	Envuelve llamadas a Firebase, SMTP y AWS SES con opossum.
Retry Pattern	Reintenta operaciones fallidas.	Backoff exponencial (1s, 2s, 4s) hasta 5 intentos para notificaciones.
Component-Pattern (Frontend)	Separa componentes presentacionales de contenedores.	ConsumptionChart (presentacional) recibe datos por props; DashboardContainer maneja estado.
Custom Hooks (Frontend)	Encapsula lógica reutilizable.	useAuth, useWebSocket, useConsumption, useNotifications.
Observer Pattern (Frontend)	Actualización en tiempo real.	El cliente consulta datos persistidos mediante polling HTTP; suscripciones WebSocket no verificadas.

Diseño de Componentes o Módulos

A continuación se describen los módulos principales del sistema Hidro Smart. Cada módulo incluye su nombre, descripción, entradas, procesos principales y salidas esperadas.

Módulo	Descripción	Entradas	Procesos principales	Salidas
Autenticación y Usuarios (RF1)	Gestiona el registro, inicio de sesión, roles, recuperación de contraseña, actualización de perfil, suspensión y eliminación de cuenta (soft delete) de los usuarios del sistema.	Correo electrónico, contraseña, datos personales, confirmación de eliminación.	Validación de credenciales, hash de contraseña con bcrypt (costo 12), generación de tokens JWT (Access + Refresh), asignación de roles (RBAC), bloqueo tras 5 intentos fallidos (15 min), envío de correo de recuperación (token expira en 1 hora), anonimización de datos al eliminar cuenta, revocación de tokens.	Token de sesión (Access + Refresh), perfil de usuario (DTO), mensajes de confirmación/error, correo de recuperación, confirmación de eliminación.
Gestión de Dispositivos IoT (RF2)	Permite registrar, calibrar, suspender, reactivar, desvincular y consultar el historial de conectividad de los medidores inteligentes (ESP32) asociados al sistema.	Código único del dispositivo (MAC/UUID), nombre, tipo, ubicación, factor K, offset, umbral de alerta (m^3/h), rango de fechas para historial.	Validación de unicidad del código, asociación al hogar en home_device, cambio de estado (activo/suspendido) con registro en device_history, almacenamiento de telemetría en device_telemetry_history (señal, batería, voltaje, temperatura), evaluación de umbral por dispositivo (prevalece sobre el del hogar), registro de calibraciones en calibration_history.	Dispositivo registrado, estado actualizado, historial de conectividad (JSON para gráficos), confirmación de calibración, mensajes de error.
Gestión de Hogares	Administra la creación,	Nombre del hogar,	Creación de hogar en home,	Hogar creado, lista de usuarios

(Proyectos) (RF3)	configuración, suspensión y reactivación de hogares, así como la asignación de usuarios titulares y miembros con funciones especiales.	dirección, ciudad, estrato (1-6), usuarios titulares y miembros, funciones especiales.	asignación de titular (Owner) y miembros (Member/Guest) en home_user, configuración de estratificación y topes por defecto según estrato en home_rate y threshold_configuration, asignación de funciones especiales en home_user_function, suspensión y reactivación con registro en audit_log.	asociados con roles, dispositivos vinculados, estado de suspensión, confirmación de asignación.
Configuraciones del Sistema (RF4)	Permite personalizar alertas, topes de consumo, recomendaciones, idioma, moneda, temas visuales, auditoría y modo vacaciones.	Topes de consumo (diario/mensual), umbrales de alerta, idioma (es/en/pt/fr), moneda (COP/USD/EUR/MXN), tema (Light/Dark/High Contrast), fechas de modo vacaciones.	Validación de valores positivos, almacenamiento en threshold_configuration, alert_rule, user_preference y vacation_mode, activación de reglas de negocio, cambio de comportamiento de alertas durante modo vacaciones (solo severidad Alta o Crítica), registro en audit_log.	Configuración guardada, alertas activadas según umbrales, preferencias aplicadas en frontend (React + Vite con i18n y Tailwind CSS), modo vacaciones activado/desactivado.
Monitoreo en Tiempo Real y Alertas (RF5)	Visualiza el consumo actual en tiempo real (polling HTTP; WebSockets no verificado), genera alertas automáticas por consumo excesivo, fugas	Datos de caudal del dispositivo IoT (litros/segundo), métricas de salud (señal, batería, voltaje, temperatura), timestamp.	Procesamiento de lecturas con calibración (factor K y offset), clasificación por actividad (ducha, lavado, cocina, aseo, etc.), comparación con topes configurados	Consumo en tiempo real (polling HTTP; WebSockets no verificado), gráfico actualizado en frontend (React + recharts), notificaciones

	o desconexión, y gestiona eventos de alerta.		(hogar y dispositivo), detección de fugas (flujo continuo >30 min), generación de alerta en alert_event, almacenamiento en device_telemetry_history, envío de notificaciones push/email/SMS vía Event Bus y colas (Bull).	push (FCM) y correos electrónicos, registro de alerta en BD, historial de conectividad actualizado.
Reportes y Análisis (RF5)	Genera informes históricos, gráficos comparativos, análisis de hábitos de consumo, predicciones y exportación a PDF y Excel.	Rango de fechas, hogar seleccionado, tipo de reporte (diario, semanal, mensual, anual), formato de exportación (PDF/Excel).	Consulta de datos históricos desde sensor_reading, daily_consumption_summary, monthly_consumption_history, generación de gráficos (recharts en frontend), cálculo de hábitos de consumo (horas pico, días de mayor consumo), aplicación de modelo predictivo (media móvil simple o regresión lineal), generación de archivos PDF (pdf-lib) y Excel (exceljs) de forma asíncrona (procesamiento asíncrono no verificado no verificado).	Reportes PDF/Excel descargables, gráficos comparativos, hábitos de consumo, predicción de consumo, análisis de tendencias.
Metas y Presupuesto (RF5.7)	Permite al usuario definir metas de ahorro de agua (en m^3) y límites de	Límite de consumo (m^3), presupuesto máximo (COP), período	Inserción/actualización en saving_goal, cálculo de progreso (porcentaje usado)	Progreso actual (porcentaje), proyección de fin de mes, estado de

	presupuesto (en COP) con seguimiento visual de progreso y proyecciones de fin de mes.	(mensual, semanal, anual), fecha de inicio y fin (opcional).	basado en consumo real vs meta, proyección de consumo final (con promedio diario), generación de alertas automáticas al alcanzar el 80% y 90% del límite, historial de metas anteriores (últimos 6 meses).	cumplimiento (alcanzada/no alcanzada), historial de metas, alertas de proximidad al límite.
Soporte Técnico (RF5.8)	Sistema de tickets para que los usuarios reporten incidencias o consultas técnicas, y el equipo de soporte gestione su resolución.	Título, categoría (Dispositivo, Reportes, Tarifas, Detección, Otros), prioridad (Baja, Media, Alta), descripción detallada, archivos adjuntos (opcional).	Inserción en ticket con estado 'Abierto', actualización de estado por soporte (En Proceso, Resuelto, Cerrado) en ticket, adición de respuestas en ticket_response, notificación al usuario por correo/push cuando se responde o resuelve, archivado automático de tickets cerrados después de 30 días, registro en audit_log.	Ticket creado con ID único, historial de respuestas, estado de resolución, notificaciones al usuario, registro en auditoría.
Modo Vacaciones (RF4.7)	Suspende notificaciones de consumo normal y ajusta alertas a solo eventos críticos (fugas, desconexión de dispositivos) durante la ausencia del usuario titular.	Fecha de inicio, fecha de fin, objetivo de consumo diario (opcional, en m^3), notificar al regreso (booleano).	Activación/desactivación en vacation_mode con active = true/false, cambio en el comportamiento de las reglas de alerta (solo se envían alertas de severidad "Alta" o "Crítica"), desactivación automática al llegar la fecha de fin, registro en	Estado del modo vacaciones (activo/inactivo), suspensión de notificaciones no críticas, mantenimiento de alertas de seguridad, confirmación de activación/desactivación.

			audit_log.	

Diagramas del Sistema (UML)

Los diagramas UML representan gráficamente el diseño del sistema Hidro Smart, facilitando la comprensión de su estructura, comportamiento y componentes. A continuación se presentan los diagramas más relevantes elaborados para este proyecto.

Diagrama de Casos de Uso

Representa los actores (Usuario, Administrador, Soporte Técnico y Sistema IoT) y los casos de uso principales del sistema. Archivo fuente: Diagramas de Casos de Uso.drawio

https://drive.google.com/drive/folders/1_S3C4k7N0oDvtXIebcFDupXPOaXKEgQU?usp=drive_link

Diagrama de Clases (Diagrama de Dominio)

Muestra las principales entidades del sistema y sus relaciones (usuarios, hogares, dispositivos, lecturas, alertas, etc.). Archivo fuente: diagrama de dominio.drawio + Copia de MER.png

https://drive.google.com/drive/folders/1KfAR3c4ENyvTRjR08R2E1Au3oeB63phK?usp=drive_link

Diagrama de Actividad

Describe el flujo detallado de los procesos principales (RF1 a RF5). Archivo fuente: Diagramas de Actividades.drawio (5 diagramas: RF1, RF2, RF3, RF4 y RF5)

https://drive.google.com/drive/folders/1lq7OcCKL2-YpipC9u8UQzUvz7BH7togz?usp=drive_link

Diagrama de Transición de Estados

Ilustra los diferentes estados por los que pueden pasar los objetos clave (sesión de usuario, dispositivo, hogar, alerta). Archivo fuente: diagrama de trancicion de estados.drawio https://drive.google.com/drive/folders/1YRPdO0aztE2tG7kAMc2_wJRs9acPrxF-?usp=drive_link

Diagrama de Secuencia

Representará el flujo temporal de mensajes entre objetos para los casos más críticos (Login, Lectura en tiempo real y Generación de alerta). Estado: Pendiente (se recomienda crear en Draw.io o PlantUML).

[Insertar imagen del Diagrama de Transición de Estados aquí]

Diagrama de Componentes

Muestra la organización física de los componentes del sistema (frontend web y shell móvil, Backend Node.js, Base de datos PostgreSQL y dispositivos IoT). Estado: Pendiente.

[Insertar imagen del Diagrama de Transición de Estados aquí]

Diagrama de Despliegue

Ilustra la infraestructura donde se ejecutarán los componentes (dispositivos IoT, servidor en la nube y dispositivos móviles del usuario). Estado: Pendiente.

[Insertar imagen del Diagrama de Transición de Estados aquí]

Diseño de Base de Datos

El diseño de la base de datos se realizó bajo el modelo relacional, garantizando la integridad de los datos, la normalización y el cumplimiento de todos los requerimientos funcionales y no funcionales definidos en el SRS v1.

Modelo Entidad - Relación

El modelo Entidad-Relación (MER) representa todas las entidades principales del sistema y sus relaciones.

Archivo fuente: Copia de MER.png

https://drive.google.com/drive/folders/10q-U9_k-FVHzPtbMjG2GP4O6VEaU28QJ?usp=drive_link

Tablas principales

Tabla	Descripción
usuarios	Información de todos los usuarios del sistema (domésticos, administradores y soporte).
roles	Definición de roles y permisos del sistema.
hogares	Entidad que representa cada hogar o proyecto registrado.
dispositivos	Registro de medidores inteligentes IoT.
lecturas (consumos)	Registro detallado de cada lectura de consumo enviada por los dispositivos.
alertas	Registro de todas las alertas generadas por el sistema.
recomendaciones	Sugerencias automáticas de ahorro generadas según los hábitos.
auditorias	Bitácora completa de todas las acciones críticas realizadas por los usuarios.
estratos	Tabla de referencia para estratos socioeconómicos y valores base.
usuarios_hogares	Relación muchos a muchos entre usuarios y

	hogares.
dispositivos_hogares	Relación muchos a muchos entre dispositivos y hogares.

Diccionario de Datos

A continuación se presenta el diccionario de datos completo del sistema Hidro Smart, organizado por módulos funcionales.

https://drive.google.com/drive/folders/1Ol3vwXOemNJgrceT57sJ9F358DzI1_ib?usp=drive_link

Diseño de Interfaces

El diseño de interfaces de Hidro Smart busca ofrecer una experiencia intuitiva, moderna y responsiva, optimizada para navegadores web y dispositivos móviles. Todos los prototipos fueron creados en Figma y siguen los principios de usabilidad y accesibilidad (WCAG AA).

https://drive.google.com/drive/folders/1Ittp-5IzEM4EQfMXPU3OtklOq3dttgvD?usp=drive_link

Tecnologías Frontend (React + Vite)

Tecnología	Versión	Propósito
React	**18.x**	**Biblioteca para construir la interfaz de usuario basada en componentes.**
Vite	**4.x**	**Bundler ultrarrápido para desarrollo y producción (HMR, builds optimizados).**
React Router	**6.x**	**Enrutamiento de la aplicación (SPA con rutas protegidas).**
Axios	**1.x**	**Cliente HTTP con interceptores para manejo de tokens JWT.**
Cliente HTTP con interceptores para manejo de tokens JWT.	**4.x**	**Comunicación en tiempo real (polling HTTP; WebSockets no verificado) para actualización del dashboard.**
Tailwind CSS	**3.x**	**Framework de CSS para estilos responsivos y personalización rápida.**
react-i18next	**13.x**	**Internacionalización (idiomas: es, en, pt, fr).**
recharts	**2.x**	**Librería de gráficos para dashboards y reportes.**
react-hook-form + zod	**7.x / 3.x**	**Manejo de formularios y validación de esquemas.**
Workbox	**7.x**	**Service Worker para PWA (modo offline parcial).**
Vitest + React Testing Library	**1.x / 14.x**	**Pruebas unitarias e integración de componentes.**

Prototipos de pantallas principales:

Autenticación y Seguridad (RF1)

Pantalla	**Pantalla**	**RF asociado**
Login	**Pantalla inicial con campos de correo electrónico y contraseña. Incluye enlace a "¿Olvidaste tu contraseña?" y botón de inicio de sesión.**	**RF1.2**
Registro	**Formulario con pestaña para crear cuenta. Campos: nombre completo, correo, contraseña, confirmación, tipo de documento y número de documento.**	**RF1.1**
Recuperar Contraseña	**Pantalla dedicada donde el usuario ingresa su correo y recibe un enlace temporal (válido por 1 hora) para restablecer su contraseña.**	**RF1.3**
Actualizar Perfil	**Formulario para modificar datos personales: nombre, teléfono, ciudad y foto de perfil (opcional). Los cambios sensibles requieren verificación por correo.**	**RF1.4**
Eliminar Cuenta	**Pantalla en "Configuración → Zona de Peligro" con doble confirmación (checkbox y verificación de contraseña) para eliminar la cuenta de forma permanente (soft delete).**	**RF1.6**

Gestión de Hogares (RF3)

Pantalla	Descripción	RF asociado
Mis Hogares	Lista de hogares asociados al usuario, con información resumida: nombre, dirección, estrato, número de dispositivos, miembros y consumo del mes. Botones	RF3.1, RF3.4

	para editar, suspender y ver detalles.	
Crear Hogar	Formulario para registrar un nuevo hogar: nombre, dirección, ciudad, estrato (selector 1-6) y tope de consumo mensual (opcional).	RF3.1
Detalle de Hogar	Vista completa del hogar: información general, lista de miembros con roles (Owner, Member, Guest), dispositivos vinculados, configuración de topes y modo vacaciones.	RF3.2, RF3.3
Gestionar Miembros	Pantalla para invitar nuevos miembros por correo electrónico, asignar roles (Member/Guest) y funciones especiales (ej. administrar dispositivos).	RF3.2.1, RF3.2.1.1

Gestión de Dispositivos IoT (RF2)

Pantalla	Descripción	RF asociado
Mis Dispositivos	Lista de sensores IoT vinculados al hogar seleccionado. Cada tarjeta muestra: nombre, estado (activo/suspendido), batería, señal, flujo actual y volumen total del día. Botones para configurar, calibrar y desvincular.	RF2.1, RF2.2, RF2.4
Registrar Dispositivo	Formulario para vincular un nuevo medidor IoT: código único (MAC/UUID), nombre descriptivo, tipo, ubicación y umbral de alerta (m^3/h, opcional).	RF2.1, RF2.5
Calibrar Dispositivo	Formulario para ajustar el factor de calibración (K) y offset del sensor. Requiere motivo de la calibración y solo está disponible para soporte técnico o administradores.	RF2.3

Historial de Conectividad	Pantalla con gráfico (recharts) que muestra la evolución de la señal, batería, voltaje y temperatura del dispositivo en un rango de fechas seleccionable (últimos 7 días, 30 días, etc.).	RF2.6

Configuraciones del Sistema (RF4)

Pantalla	Descripción	RF asociado
Topes de Consumo	Configuración de límites máximos de consumo (diario y mensual) para el hogar. Validación de valores positivos.	RF4.1
Alertas	Configuración de reglas de alerta personalizadas: tipo (consumo excesivo, fuga, tope mensual, etc.), umbral, unidad y activación.	RF4.2
Recomendaciones	Configuración de la generación automática de recomendaciones (activar/desactivar).	RF4.3
Idioma y Moneda	Selectores para cambiar el idioma (es, en, pt, fr) y la moneda (COP, USD, EUR, MXN).	RF4.4, RF4.4.1
Temas Visuales	Selección de tema (Light, Dark, High Contrast) con previsualización en tiempo real.	RF4.5
Auditoría	Panel de consulta de logs de auditoría (solo administradores). Filtros por usuario, fecha y tipo de acción.	RF4.6
Modo Vacaciones	Pantalla para activar/desactivar el modo vacaciones con selección de fechas de inicio y fin, objetivo de consumo diario (opcional) y opción de notificar al regreso.	RF4.7

Monitoreo, Reportes y Análisis (RF5)

Pantalla	Descripción	RF asociado
Dashboard (Inicio)	Panel principal con: consumo actual en tiempo real (gauge), flujo actual (m³/h), consumo del día/semana/mes, alertas activas, estado de dispositivos y recomendaciones destacadas. Actualización vía polling HTTP; WebSockets no verificado.	RF5.1.1, RF5.4, RF5.5
Consumo en Tiempo Real	Pantalla con gráfico de consumo en vivo (recharts) y desglose por actividad (ducha, lavado, cocina, aseo, etc.).	RF5.1.1
Reportes y Análisis	Selector de período (diario, semanal, mensual, anual), gráficos comparativos (barras, líneas, circulares), análisis de hábitos (horas pico, días de mayor consumo), proyección de costos y botones de exportación a PDF y Excel.	RF5.1, RF5.2, RF5.3
Metas y Presupuesto	Configuración de metas de consumo (m³) y presupuesto (COP) con seguimiento visual de progreso (barras de porcentaje), proyección de fin de mes e historial de metas anteriores.	RF5.7
Notificaciones / Alertas	Lista de eventos de alerta con filtros por estado (todas, no leídas, leídas, descartadas). Cada alerta muestra tipo, severidad, mensaje, valor detectado y fecha. Opción de marcar como leída o descartada.	RF5.4
Soporte Técnico	Gestión de tickets: listado con filtros (todos, abiertos, en proceso, cerrados), creación	RF5.8

	de nuevos tickets (título, categoría, prioridad, descripción, archivos adjuntos), visualización de detalle y respuestas.	

Formularios principales

Formulario	Campos	Validaciones	RF asociado
Registro de usuario	Nombre completo, correo electrónico, contraseña, confirmación, tipo de documento, número de documento.	Correo único, contraseña ≥8 caracteres con mayúscula, minúscula, número y símbolo.	RF1.1
Recuperación de contraseña	Correo electrónico.	Correo registrado en el sistema (sin revelar si existe o no por seguridad).	RF1.3
Actualización de perfil	Nombre completo, teléfono, ciudad, foto de perfil (opcional).	Teléfono con formato válido, foto ≤2MB (JPG/PNG).	RF1.4
Creación de hogar	Nombre del hogar, dirección, ciudad, estrato (1-6).	Nombre único para el usuario, estrato entre 1 y 6.	RF3.1
Vinculación de dispositivo	Código único (MAC/UUID), nombre, tipo, ubicación, umbral de alerta (opcional).	Código único en el sistema, umbral > 0 (si se define).	RF2.1, RF2.5
Calibración de dispositivo	Factor K, offset, motivo de calibración.	Factor K > 0, motivo obligatorio.	RF2.3
Configuración de alertas	Tipo de alerta, umbral, unidad, activa/inactiva.	Umbral > 0, unidad coherente con el tipo.	RF4.2
Modo vacaciones	Fecha de inicio, fecha de fin, objetivo de consumo diario (opcional), notificar al regreso.	Fecha de fin ≥ fecha de inicio, objetivo ≥ 0 (si se define).	RF4.7
Meta de ahorro	Límite de consumo (m^3), presupuesto (COP), período (mensual/semanal/anual).	Límite > 0, presupuesto > 0.	RF5.7
Creación de ticket	Título, categoría, prioridad, descripción,	Título y descripción obligatorios,	RF5.8

	archivos adjuntos (opcional).	categoría y prioridad predefinidas.	

Menús y navegación (React Router)

Elemento	Descripción	Rutas asociadas
Menú lateral (Drawer)	Navegación principal para versiones de escritorio y tabletas.	/, /homes, /devices, /reports, /goals, /notifications, /support, /settings
Menú inferior (Bottom Navigation)	Navegación para dispositivos móviles (pantallas pequeñas).	Inicio, Hogares, Dispositivos, Reportes, Configuración
Acceso rápido a Alertas	Icono de campana en el header, visible desde cualquier pantalla.	/notifications
Perfil de usuario	Acceso desde el menú lateral o avatar en el header.	/settings/profile
Panel de administración	Solo visible para usuarios con rol "Administrador" o "Soporte Técnico".	/admin/* (auditoría, gestión de usuarios, etc.)

Flujo de navegación principal:

1. **Login → Dashboard (redirección tras autenticación exitosa).**
2. **Dashboard → Menú lateral/inferior → Cualquier pantalla (Hogares / Dispositivos / Reportes / Metas / Soporte / Configuración).**
3. Desde cualquier pantalla → Acceso rápido a Alertas (icono de campana en el header).
4. **Rutas protegidas: Todas las rutas excepto /login, /register, /forgot-password, /reset-password requieren autenticación (middleware de React Router).**

Estilo y diseño visual

Aspecto	Descripción
Paleta de colores	Colores institucionales (azules y verdes) que transmiten confianza, sostenibilidad y tecnología.
Tipografía	Fuente sans-serif (Inter o Roboto) para legibilidad en pantallas.

Responsividad	Diseño mobile-first con Tailwind CSS. Adaptación a resoluciones desde 320px (móviles) hasta 1920px (escritorio).
Accesibilidad	Cumplimiento de WCAG 2.1 nivel AA: contraste ≥ 4.5:1, navegación por teclado, etiquetas ARIA, texto alternativo en imágenes.
Temas	Soporte para modo claro, oscuro y alto contraste, con persistencia en user_preference.
Feedback visual	Animaciones suaves, estados de carga (Skeleton o Spinner), notificaciones tipo Toast/SnackBar para confirmaciones y errores.

Reglas de negocio

Las reglas de negocio definen las condiciones, restricciones y políticas que debe cumplir el sistema Hidro Smart para garantizar su correcto funcionamiento, integridad de datos y cumplimiento de los requerimientos funcionales establecidos en el SRS v1.1 (RF1 a RF11).

A continuación se detallan las reglas principales, organizadas por módulos funcionales:

Reglas de Autenticación y Seguridad (RF1)

#	Regla	Descripción
1	Unicidad de identidad	Un usuario no puede registrarse con un correo electrónico o número de documento ya existente en el sistema.
2	Contraseña segura	La contraseña debe tener mínimo 8 caracteres e incluir al menos una mayúscula, una minúscula, un número y un

		símbolo; se almacena siempre cifrada en la base de datos (bcrypt, costo 12).
3	Bloqueo por intentos fallidos	Tras 5 intentos fallidos de inicio de sesión consecutivos, la cuenta se bloquea temporalmente por 15 minutos.
4	Recuperación de contraseña	El enlace de recuperación de contraseña es válido únicamente por 1 hora y es de un solo uso.
5	Eliminación de cuenta (soft delete)	Al eliminar una cuenta, los datos personales se anonimizan y todos los tokens JWT son revocados. El historial de consumo se conserva anonimizado para estadísticas.
6	Actualización de perfil	Los cambios en información sensible requieren verificación por correo electrónico antes de ser aplicados.

Reglas de Gestión de Hogares (RF3)

#	Regla	Descripción
7	Titular único por hogar	Solo puede existir un usuario titular (Owner) por hogar; los demás miembros pueden tener roles Member o Guest.
8	Asignación de miembros	Un usuario solo puede ser invitado a un hogar si ya está registrado. No se permiten duplicados en la relación usuario-hogar.

9	Suspensión de hogar	Un hogar suspendido no genera alertas ni notificaciones hasta ser reactivado. Las lecturas continúan almacenándose.
10	Reactivación de hogar	Un hogar suspendido solo puede ser reactivado por el titular o un administrador.

Reglas de Gestión de Dispositivos IoT (RF2)

#	Regla	Descripción
11	Código único de dispositivo	El código identificador (MAC/UUID) de cada dispositivo IoT debe ser único en todo el sistema.
12	Vinculación exclusiva	Un dispositivo IoT solo puede estar vinculado a un único hogar al mismo tiempo.
13	Registro condicional de lecturas	El consumo solo se registra si el dispositivo está activo y el hogar no está suspendido.
14	Calibración restringida	La calibración solo puede ser realizada por usuarios con rol Soporte Técnico o Administrador.
15	Umbral por dispositivo	El umbral individual del dispositivo prevalece sobre el umbral del hogar; si no existe, se aplica el del hogar o el valor por defecto según estrato.
16	Desvinculación de dispositivo	Un dispositivo puede desvincularse de un hogar y ser reasignado posteriormente conservando su historial.

Reglas de Configuraciones del Sistema (RF4)

#	Regla	Descripción
17	Consistencia de topes	Los topes de consumo deben ser positivos y al menos uno (diario o mensual) debe estar definido.

18	Reglas de alerta activas	Solo las reglas marcadas como activas generan eventos de alerta.
19	Modo vacaciones	Durante el modo vacaciones solo se notifican alertas críticas o altas; el modo se desactiva automáticamente al finalizar el período.
20	Auditoría obligatoria	Toda acción crítica debe registrarse en la tabla audit_log.

Reglas de Monitoreo, Alertas y Reportes (RF5)

#	Regla	Descripción
21	Generación de alertas	Se genera una alerta cuando el consumo supera el tope en ≥10%, se detecta flujo continuo por más de 30 minutos o un sensor permanece sin enviar datos durante más de una hora.
22	Acceso a reportes	Los reportes solo pueden generarse para hogares asociados al usuario autenticado.
23	Recomendaciones personalizadas	Las recomendaciones se generan con datos históricos reales y no se repiten antes de 7 días. Se ejecutan diariamente a las 6:00 AM.
24	Metas de ahorro	Se generan alertas al alcanzar el 80% y 90% de la meta de ahorro. La proyección mensual utiliza el promedio de consumo de los últimos siete días.
25	Historial de conectividad	El historial de señal, batería, voltaje y temperatura se conserva durante al menos 30 días.
26	Reportes asíncronos	La generación de reportes PDF/Excel se realiza mediante mecanismos

		asíncronos no verificados para evitar bloquear la API.

Reglas de Soporte Técnico (RF5.8)

#	Regla	Descripción
27	Categorías y prioridades	Los tickets deben tener una categoría y una prioridad predefinidas.
28	Gestión de estados	Los estados válidos son: Abierto, En Proceso, Resuelto y Cerrado. Solo soporte o administradores pueden modificarlos.
29	Archivado automático	Los tickets cerrados se archivan automáticamente tras 30 días sin actividad.
30	Notificaciones de tickets	El usuario recibe una notificación cuando su ticket es respondido o cambia a Resuelto o Cerrado.

Reglas Transversales de Seguridad y Operación

#	Regla	Descripción
31	Protección de datos personales	El tratamiento de datos cumple con la Ley 1581 de 2012. El usuario debe otorgar consentimiento y puede ejercer sus derechos ARCO.
32	Tolerancia a fallos	Los servicios externos implementan Circuit Breaker y Retry Pattern. Tras tres fallos consecutivos se abre el

		circuito durante 30 segundos y se registra el evento.
33	Procesamiento asíncrono	Las tareas pesadas se ejecutan mediante mecanismos asíncronos no verificados para mantener tiempos de respuesta inferiores a 500 ms.
34	Escalabilidad horizontal	El backend es stateless y utiliza JWT + Redis no verificado para la gestión de sesiones, permitiendo escalar horizontalmente.

Aplicación de las Reglas

Capa	Mecanismo de aplicación
Frontend (React + Vite)	Validaciones en formularios con react-hook-form y zod, mensajes de error en tiempo real y deshabilitación de botones en acciones críticas.
Backend (Node.js)	Validaciones con express-validator, lógica de negocio en arquitectura hexagonal y manejo estandarizado de excepciones HTTP.
Base de datos (PostgreSQL)	Restricciones NOT NULL, UNIQUE, CHECK, llaves foráneas, triggers para updated_at y validaciones adicionales.
Auditoría	Middleware que registra acciones críticas en audit_log con usuario, IP, user-agent y valores anteriores y nuevos.

Seguridad del Sistema

Seguridad del Sistema

El sistema Hidro Smart implementa múltiples capas de seguridad para proteger la información de los usuarios, garantizar la integridad de los datos y cumplir con la normativa colombiana (Ley 1581 de 2012) y estándares internacionales (ISO/IEC 27001). Los mecanismos de protección se aplican en todas las capas del sistema (frontend, backend y base de datos).

Autenticación

El sistema utiliza JWT (JSON Web Tokens) con firma HS256 para la autenticación stateless. Se generan dos tipos de tokens: el Access Token tiene una duración corta de 15 minutos y se almacena en memoria (React) o en una cookie HttpOnly según la configuración. El Refresh Token tiene una duración extendida de 7 días, se almacena en la base de datos (tabla token) y en el cliente (almacenamiento seguro), permitiendo renovar el Access Token sin volver a iniciar sesión.

El frontend (React + Vite) utiliza un interceptor de Axios para renovar automáticamente el Access Token cuando expira, sin intervención del usuario. Los tokens pueden ser revocados individualmente (cierre de sesión) o en masa (cambio de contraseña, eliminación de cuenta). Para garantizar la invalidación inmediata, se utiliza Redis no verificado como blacklist para tokens revocados.

La autenticación de dos factores (2FA) está prevista para versiones futuras, utilizando TOTP vía Google Authenticator o SMS. La estructura de la base de datos ya incluye la tabla user_mfa para este propósito. Además, se implementa un bloqueo temporal de cuenta durante 15 minutos después de 5 intentos fallidos consecutivos, y el contador se reinicia tras un inicio de sesión exitoso.

Autorización (RBAC)

Se implementa RBAC (Role-Based Access Control) con tres roles principales. El Usuario doméstico tiene permisos de CRUD sobre sus propios hogares y dispositivos, consulta de reportes, configuración de alertas y topes, gestión de metas, modo vacaciones, tickets de soporte, actualización de perfil y eliminación de cuenta propia. El Administrador tiene permisos de CRUD sobre todos los hogares, usuarios y dispositivos; acceso a auditorías (audit_log); suspensión y reactivación de cuentas; gestión de roles y permisos; y configuración global del sistema. El Soporte Técnico tiene permisos para gestionar dispositivos (calibración, suspensión, reactivación), consultar el historial de conectividad y gestionar tickets de soporte, pero no tiene acceso a datos personales de usuarios ni a configuración de hogares.

Cada endpoint de la API verifica el token JWT y el rol mediante middleware (auth.middleware.js y role.middleware.js). Además, las políticas de acceso se aplican también a nivel de base de datos mediante RLS (Row Level Security) en PostgreSQL.

Control de acceso

Todas las rutas sensibles están protegidas por middleware de autenticación. Las rutas públicas son solo /login, /register, /forgot-password y /reset-password. El control de acceso basado en el hogar garantiza que un usuario solo pueda ver y modificar los datos de los hogares a los que está vinculado (Owner, Member o Guest). Esto se aplica en la capa de servicio (casos de uso) y se refuerza con RLS en PostgreSQL.

Se implementa un límite de 100 solicitudes por minuto por usuario (basado en IP o token JWT) para prevenir ataques de fuerza bruta y DDoS. Además, se registra en la tabla audit_log toda

acción crítica: quién (usuario), qué (acción), cuándo (timestamp), desde qué IP (source_ip) y qué valores cambiaron (old_value y new_value en JSONB).

Protección de datos

Todas las comunicaciones se realizan mediante HTTP local; HTTPS/TLS productivo pendiente para API y MQTT local sin TLS; TLS/ACL de producción pendiente para la comunicación con los sensores IoT. La base de datos PostgreSQL se cifra a nivel de disco mediante AES-256 y los campos sensibles (contraseñas, tokens) se almacenan hasheados o cifrados.

Para proteger el sistema contra inyección SQL, se utilizan exclusivamente consultas parametrizadas (prepared statements) en todos los repositorios (PostgreSQL). Los tokens JWT en el frontend se almacenan en memoria (React state) o en cookies HttpOnly para mayor seguridad contra ataques XSS.

Las copias de seguridad se realizan automáticamente a diario, se cifran con AES-256 y se almacenan en almacenamiento externo no verificado con acceso restringido. El sistema cumple con la Ley 1581 de 2012 (protección de datos personales en Colombia), donde el usuario debe dar consentimiento explícito (user_consent) y puede ejercer sus derechos ARCO (acceso, rectificación, cancelación, oposición, portabilidad) a través del módulo correspondiente.

Además, al eliminar una cuenta (RF1.6), los datos personales se anonimizan: el nombre se reemplaza por "Usuario eliminado", el correo por deleted_{id}@hidrosmart.com y el documento por 0000000000. El historial de consumo se conserva anonimizado para estadísticas agregadas

Manejo de contraseñas

Las contraseñas nunca se almacenan en texto plano. Se utiliza bcrypt con factor de costo 12 para generar el hash. Se aplican reglas de complejidad según la password_policy definida: mínimo 8 caracteres, al menos una mayúscula, una minúscula, un número y un símbolo.

La recuperación de contraseña se realiza mediante un enlace temporal enviado al correo electrónico, válido por 1 hora. El enlace contiene un token de un solo uso almacenado en password_reset_token. Además, se ofrece la opción de cambio obligatorio de contraseña en el primer inicio de sesión (si fue generada por el administrador) y no se permite reutilizar las últimas 5 contraseñas (opcional, según política de seguridad).

Tolerancia a fallos y resiliencia

El sistema implementa Circuit Breaker con la librería opossum para servicios externos como servicio de notificaciones no verificado, envío de emails y SMS. Si un servicio falla 3 veces consecutivas, se "abre" el circuito y se omite la llamada por 30 segundos, registrando el evento en log_error. También se aplica el Retry Pattern: las operaciones fallidas (envío de notificaciones, sincronización con servicios externos) se reintentan con backoff exponencial (1s, 2s, 4s, hasta 5 intentos).

Las tareas que pueden fallar (envío de emails, notificaciones push, generación de reportes) se procesan de forma asíncrona mediante mecanismos asíncronos no verificados para no bloquear la API principal. Además, se implementa un middleware global de errores en Express que captura excepciones, las registra en log_error y devuelve respuestas HTTP estandarizadas (400, 401, 403, 404, 500) sin exponer stack traces en producción.

9.7 Seguridad en el frontend (React + Vite)

Los tokens JWT se almacenan en memoria (React state) o en cookies HttpOnly para mayor seguridad contra ataques XSS. Si se utilizan cookies, se implementa protección CSRF con tokens de sincronización o usando el atributo SameSite=Strict. Todos los formularios validan las entradas del usuario tanto en el frontend (con react-hook-form + zod) como en el backend (con express-validator). Las entradas del usuario se sanitizan para prevenir XSS (Cross-Site Scripting) usando librerías como DOMPurify en el frontend y validator en el backend. Además, se implementa Content Security Policy (CSP) para restringir la ejecución de scripts no autorizados y la aplicación se sirve siempre a través de HTTPS, redirigiendo automáticamente HTTP a HTTPS.

Resumen de mecanismos por capa

En el frontend (React + Vite) , la seguridad se aplica mediante almacenamiento seguro de tokens (memoria o cookies HttpOnly), validación de formularios (react-hook-form + zod), sanitización de entradas (DOMPurify), Content Security Policy (CSP) y HTTPS forzado.

En el backend (Node.js) , se utilizan JWT con Access/Refresh tokens, bcrypt (costo 12), middleware de autenticación y roles, rate limiting, Circuit Breaker, Retry Pattern, procesamiento asíncrono con mecanismos asíncronos no verificados, manejo centralizado de errores y logs en log_error.

En la base de datos (PostgreSQL) , se aplica cifrado en reposo (AES-256), RLS (Row Level Security), consultas parametrizadas (prepared statements), restricciones de integridad (NOT NULL, UNIQUE, CHECK) y backups cifrados diarios.

En la infraestructura, se garantiza HTTP local; HTTPS/TLS productivo pendiente en todas las comunicaciones, MQTT local sin TLS; TLS/ACL de producción pendiente, almacenamiento

externo no verificado con acceso restringido para backups y reportes, y Redis no verificado con autenticación para colas y blacklist.

Especificaciones Técnicas

A continuación, se detallan las tecnologías seleccionadas para el desarrollo de Hidro Smart, garantizando alto rendimiento, escalabilidad, seguridad y facilidad de mantenimiento.

Componente	Tecnología utilizada	Versión	Propósito / Justificación
Frontend	React + Vite	18.x / 4.x	Biblioteca para construir interfaces de usuario basadas en componentes. Vite como bundler ultrarrápido con HMR y builds optimizados.
Enrutamiento	React Router	6.x	Navegación SPA con rutas protegidas y lazy loading.
Cliente HTTP	Axios	1.x	Cliente HTTP con interceptores para manejo de tokens JWT y renovación automática.
polling HTTP; WebSockets no verificado	cliente HTTP	4.x	Comunicación bidireccional en tiempo real para actualización del dashboard.
Estilos	Tailwind CSS	3.x	Framework de CSS para diseño responsivo, personalizable y mobile-first.
Internacionalización	react-i18next	13.x	Soporte para múltiples idiomas (es, en, pt, fr) con

			traducciones dinámicas.
Gráficos	recharts	2.x	Librería de gráficos para dashboards y reportes (barras, líneas, circulares).
Formularios y validación	react-hook-form + zod	7.x / 3.x	Manejo eficiente de formularios y validación de esquemas con tipos.
PWA (Offline)	Workbox	7.x	Service Worker para modo offline parcial y caché de recursos estáticos.
Pruebas frontend	Vitest + React Testing Library	1.x / 14.x	Pruebas unitarias e integración de componentes y hooks.
Backend	Node.js + Express	18 LTS / 4.x	Entorno JavaScript asíncrono, ideal para tiempo real y múltiples conexiones IoT. Express como framework web ligero y flexible.
Arquitectura	Hexagonal (Puertos y Adaptadores) + DDD	-	Separa el núcleo (dominio + casos de uso) de los detalles externos (BD, HTTP, MQTT). Facilita pruebas y mantenimiento.
polling HTTP; WebSockets no verificado (backend)	WebSocket push no verificado	4.x	Comunicación bidireccional en tiempo real con el frontend.
Base de datos	PostgreSQL	15+	Relacional robusta, soporte JSONB, RLS (Row Level Security), licencia gratuita.
Cliente BD	pg	8.x	Cliente para Node.js con consultas parametrizadas (prepared statements).

Comunicación IoT	MQTT (Mosquitto/MQTT local; cloud no verificado)	2.0 / -	Protocolo ligero y eficiente para dispositivos de bajo consumo (ESP32). TLS para seguridad.
Colas y procesamiento asíncrono	procesamiento asíncrono no verificado no verificado	4.x / 7.x	Procesamiento de tareas pesadas (reportes, notificaciones) en segundo plano sin bloquear la API.
Circuit Breaker	opossum	8.x	Tolerancia a fallos en servicios externos (Firebase, emails, SMS).
Autenticación	JWT (jsonwebtoken) + bcrypt	9.x / 5.x	Autenticación stateless con Access/Refresh tokens. Hash de contraseñas con bcrypt (costo 12).
Logging	winston	3.x	Logs estructurados en archivos y consola (niveles: error, warn, info, debug).
Validación	express-validator + Joi	7.x / 17.x	Validación de datos de entrada en controladores y casos de uso.
Pruebas backend	Jest + Supertest	29.x	Pruebas unitarias e integración de API.
Notificaciones push	servicio de notificaciones no verificado (FCM)	API v1	Entrega instantánea y confiable de alertas a dispositivos móviles.
Generación de reportes	pdf-lib + exceljs	Últimas	Exportación nativa a PDF y Excel desde el backend.
Control de versiones	Git + GitHub	-	Control de cambios, ramas, pull requests e integración continua.
Entorno de desarrollo	Visual Studio Code	Último	Editor con integración Git, depuración y extensiones para React y Node.js.

Despliegue backend	Railway / Render	-	Despliegue escalable con integración continua.
Despliegue frontend	Vercel / Netlify	-	Hosting optimizado para aplicaciones React.
Distribución móvil	Google Play Console	-	Publicación oficial de la PWA como aplicación Android.
CI/CD	GitHub Actions	-	Automatización de pruebas, linting y despliegues.

Consideraciones de Implementación

Esta sección describe los aspectos prácticos y técnicos que se tendrán en cuenta durante la fase de construcción del sistema Hidro Smart, con el fin de garantizar calidad, mantenibilidad, seguridad y cumplimiento de los estándares del SENA.

Estándares de codificación

Capa	Estándar	Herramientas	Detalles
Frontend (React + Vite)	ESLint + Prettier	ESLint (configuración recomendada React), Prettier	camelCase para variables/funciones, PascalCase para componentes/hooks. JSDoc para funciones complejas. Estructura por dominios funcionales.
Backend (Node.js)	ESLint + Prettier	ESLint (Airbnb), Prettier	camelCase para variables/funciones, PascalCase para clases, UPPER_SNAKE_CASE para constantes. JSDoc para funciones públicas. Arquitectura hexagonal.
Base de datos (PostgreSQL)	snake_case	-	snake_case para tablas y columnas. PK: id_nombre (ej. user_account_id). FK: fk_tabla_origen_tabla_destino. Índices para claves foráneas y columnas de búsqueda frecuente. COMMENT en tablas y funciones.

Git (Commits)	Conventional Commits	-	Formato: tipo(ámbito): descripción. Tipos: feat, fix, docs, style, refactor, test, chore. Commits atómicos y descriptivos.

Manejo de errores

Capa	Mecanismo	Herramientas	Detalles
Backend	**Centralizado + Middleware global**	**AppError (clase), Winston, Middleware de errores**	**Respuestas HTTP estandarizadas { success: false, message, error }. Logs en logs/error.log y logs/combined.log. Stack traces solo en desarrollo. Registro en log_error.**
Frontend	**Error Boundaries + Try-catch**	**React Error Boundaries, SnackBars/Toasts**	**Captura de errores en componentes. Mensajes amigables al usuario. Logs en consola (desarrollo) o servicio de monitoreo (producción).**
Base de datos	**Constraints SQL + Validación en aplicación**	**NOT NULL, UNIQUE, CHECK, llaves foráneas**	**Validaciones antes de insertar/actualizar. Mensajes de error claros para violaciones de integridad.**

Otras consideraciones

Aspecto	Detalle
Control de versiones	


```
hidro-smart-frontend/
├── public/
│   ├── favicon.ico
│   └── manifest.json
├── src/
│   ├── api/
│   │   ├── axiosConfig.js
│   │   ├── authApi.js
│   │   ├── homeApi.js
│   │   ├── deviceApi.js
│   │   ├── consumptionApi.js
│   │   ├── alertApi.js
│   │   ├── reportApi.js
│   │   ├── goalApi.js
```

```
│   │   ├── ticketApi.js
│   │   └── vacationApi.js
│   ├── components/
│   │   ├── common/
│   │   │   ├── Button.jsx
│   │   │   ├── Card.jsx
│   │   │   ├── Input.jsx
│   │   │   ├── Select.jsx
│   │   │   ├── Modal.jsx
│   │   │   └── LoadingSpinner.jsx
│   │   ├── layout/
│   │   │   ├── Sidebar.jsx
│   │   │   ├── Header.jsx
│   │   │   └── Footer.jsx
│   │   ├── dashboard/
│   │   │   ├── ConsumptionChart.jsx
│   │   │   ├── AlertCard.jsx
│   │   │   ├── DeviceStatusCard.jsx
│   │   │   └── RecommendationCard.jsx
│   │   ├── homes/
│   │   │   ├── HomeList.jsx
│   │   │   ├── HomeForm.jsx
│   │   │   ├── HomeDetail.jsx
```

```
│   │   │   └── MemberList.jsx
│   │   ├── devices/
│   │   │   ├── DeviceList.jsx
│   │   │   ├── DeviceForm.jsx
│   │   │   ├── DeviceDetail.jsx
│   │   │   ├── CalibrationForm.jsx
│   │   │   └── ConnectivityHistory.jsx
│   │   ├── reports/
│   │   │   ├── ReportGenerator.jsx
│   │   │   ├── ReportList.jsx
│   │   │   ├── ConsumptionCharts.jsx
│   │   │   └── CostProjection.jsx
│   │   ├── goals/
│   │   │   ├── GoalList.jsx
│   │   │   ├── GoalForm.jsx
│   │   │   └── GoalProgress.jsx
│   │   ├── tickets/
│   │   │   ├── TicketList.jsx
│   │   │   ├── TicketForm.jsx
│   │   │   ├── TicketDetail.jsx
│   │   │   └── TicketResponse.jsx
│   │   ├── settings/
│   │   │   ├── Profile.jsx
```

```
│   │   │   ├── Security.jsx
│   │   │   ├── Privacy.jsx
│   │   │   ├── Preferences.jsx
│   │   │   ├── VacationMode.jsx
│   │   │   └── Account.jsx
│   │   └── auth/
│   │       ├── Login.jsx
│   │       ├── Register.jsx
│   │       ├── ForgotPassword.jsx
│   │       └── ResetPassword.jsx
│   ├── contexts/
│   │   ├── AuthContext.jsx
│   │   ├── ThemeContext.jsx
│   │   ├── NotificationContext.jsx
│   │   └── SocketContext.jsx
│   ├── hooks/
│   │   ├── useAuth.js
│   │   ├── useWebSocket.js
│   │   ├── useConsumption.js
│   │   └── useNotifications.js
│   ├── pages/
│   │   ├── DashboardPage.jsx
│   │   ├── HomesPage.jsx
```

```
│   │   ├── DevicesPage.jsx
│   │   ├── ReportsPage.jsx
│   │   ├── GoalsPage.jsx
│   │   ├── NotificationsPage.jsx
│   │   ├── SupportPage.jsx
│   │   └── SettingsPage.jsx
│   ├── services/
│   │   ├── socketService.js
│   │   ├── notificationService.js
│   │   └── storageService.js
│   ├── styles/
│   │   ├── index.css
│   │   └── tailwind.css
│   ├── utils/
│   │   ├── validators.js
│   │   ├── formatters.js
│   │   ├── constants.js
│   │   └── helpers.js
│   ├── App.jsx
│   ├── main.jsx
│   └── router.jsx
├── .env
├── .env.example
```

```
├── .gitignore
├── index.html
├── package.json
├── tailwind.config.js
├── vite.config.js
└── README.md
```

Backend (Node.js con arquitectura hexagonal):

```
hidro-smart-backend/
├── src/
│   ├── api/
│   │   ├── controllers/
│   │   │   ├── auth.controller.js
│   │   │   ├── user.controller.js
│   │   │   ├── home.controller.js
│   │   │   ├── device.controller.js
│   │   │   ├── consumption.controller.js
│   │   │   ├── alert.controller.js
│   │   │   ├── report.controller.js
│   │   │   ├── recommendation.controller.js
│   │   │   ├── vacation.controller.js
│   │   │   ├── goal.controller.js
│   │   │   ├── ticket.controller.js
```

```
│   │   │   └── arco.controller.js
│   │   ├── middlewares/
│   │   │   ├── auth.middleware.js
│   │   │   ├── role.middleware.js
│   │   │   ├── validation.middleware.js
│   │   │   ├── audit.middleware.js
│   │   │   ├── rate-limit.middleware.js
│   │   │   └── error-handler.middleware.js
│   │   ├── routes/
│   │   │   └── v1/
│   │   │       ├── index.js
│   │   │       ├── auth.routes.js
│   │   │       ├── user.routes.js
│   │   │       ├── home.routes.js
│   │   │       ├── device.routes.js
│   │   │       ├── consumption.routes.js
│   │   │       ├── alert.routes.js
│   │   │       ├── report.routes.js
│   │   │       ├── recommendation.routes.js
│   │   │       ├── vacation.routes.js
│   │   │       ├── goal.routes.js
│   │   │       ├── ticket.routes.js
│   │   │       └── arco.routes.js
```

```
│   │   └── validators/
│   │       ├── auth.validator.js
│   │       ├── user.validator.js
│   │       ├── home.validator.js
│   │       └── ...
│   ├── core/
│   │   ├── domain/
│   │   │   ├── entities/
│   │   │   │   ├── User.js
│   │   │   │   ├── Home.js
│   │   │   │   ├── Device.js
│   │   │   │   ├── Reading.js
│   │   │   │   ├── Alert.js
│   │   │   │   ├── VacationMode.js
│   │   │   │   └── Goal.js
│   │   │   ├── value-objects/
│   │   │   │   ├── Email.js
│   │   │   │   ├── Password.js
│   │   │   │   ├── DeviceCode.js
│   │   │   │   └── Consumption.js
│   │   │   ├── events/
│   │   │   │   ├── ReadingCreatedEvent.js
│   │   │   │   ├── AlertTriggeredEvent.js
```

```
│   │   │   │   └── DeviceStatusChangedEvent.js
│   │   │   └── exceptions/
│   │   │       ├── DomainException.js
│   │   │       ├── ValidationException.js
│   │   │       ├── NotFoundException.js
│   │   │       └── UnauthorizedException.js
│   │   ├── application/
│   │   │   ├── use-cases/
│   │   │   │   ├── auth/
│   │   │   │   │   ├── RegisterUserUseCase.js
│   │   │   │   │   ├── LoginUserUseCase.js
│   │   │   │   │   ├── ForgotPasswordUseCase.js
│   │   │   │   │   └── ResetPasswordUseCase.js
│   │   │   │   ├── home/
│   │   │   │   │   ├── CreateHomeUseCase.js
│   │   │   │   │   ├── UpdateHomeUseCase.js
│   │   │   │   │   ├── DeleteHomeUseCase.js
│   │   │   │   │   └── InviteMemberUseCase.js
│   │   │   │   ├── device/
│   │   │   │   │   ├── RegisterDeviceUseCase.js
│   │   │   │   │   ├── CalibrateDeviceUseCase.js
│   │   │   │   │   ├── SuspendDeviceUseCase.js
│   │   │   │   │   └── UnlinkDeviceUseCase.js
```

```
│   │   │   ├── consumption/
│   │   │   │   ├── ProcessReadingUseCase.js
│   │   │   │   ├── GetConsumptionHistoryUseCase.js
│   │   │   │   └── GeneratePredictionUseCase.js
│   │   │   ├── alert/
│   │   │   │   ├── EvaluateAlertRulesUseCase.js
│   │   │   │   ├── GetAlertsUseCase.js
│   │   │   │   └── MarkAlertAsReadUseCase.js
│   │   │   ├── vacation/
│   │   │   │   ├── ActivateVacationModeUseCase.js
│   │   │   │   └── GetVacationModeUseCase.js
│   │   │   ├── goal/
│   │   │   │   ├── CreateGoalUseCase.js
│   │   │   │   ├── UpdateGoalUseCase.js
│   │   │   │   └── DeleteGoalUseCase.js
│   │   │   └── user/
│   │   │       ├── UpdateProfileUseCase.js
│   │   │       └── DeleteAccountUseCase.js
│   │   ├── ports/
│   │   │   ├── repositories/
│   │   │   │   ├── IUserRepository.js
│   │   │   │   ├── IHomeRepository.js
│   │   │   │   ├── IDeviceRepository.js
```

```
│   │   │   │   │   ├── IReadingRepository.js
│   │   │   │   │   ├── IAlertRepository.js
│   │   │   │   │   ├── IVacationRepository.js
│   │   │   │   │   └── IGoalRepository.js
│   │   │   │   └── services/
│   │   │   │       ├── INotificationService.js
│   │   │   │       ├── IEmailService.js
│   │   │   │       ├── IPushService.js
│   │   │   │       ├── IEventBus.js
│   │   │   │       └── IUnitOfWork.js
│   │   │   └── dtos/
│   │   │       ├── requests/
│   │   │       │   ├── RegisterUserRequest.js
│   │   │       │   ├── LoginRequest.js
│   │   │       │   ├── CreateHomeRequest.js
│   │   │       │   └── ...
│   │   │       └── responses/
│   │   │           ├── AuthResponse.js
│   │   │           ├── UserProfileResponse.js
│   │   │           ├── HomeResponse.js
│   │   │           ├── DeviceResponse.js
│   │   │           └── ...
│   │   └── infrastructure/
```

```
│   │   ├── repositories/
│   │   │   ├── postgres/
│   │   │   │   ├── PostgresUserRepository.js
│   │   │   │   ├── PostgresHomeRepository.js
│   │   │   │   ├── PostgresDeviceRepository.js
│   │   │   │   ├── PostgresReadingRepository.js
│   │   │   │   ├── PostgresAlertRepository.js
│   │   │   │   ├── PostgresVacationRepository.js
│   │   │   │   ├── PostgresGoalRepository.js
│   │   │   │   ├── PostgresUnitOfWork.js
│   │   │   │   └── mappers/
│   │   │   │       ├── UserMapper.js
│   │   │   │       ├── HomeMapper.js
│   │   │   │       └── ...
│   │   │   └── in-memory/
│   │   │       └── ...
│   │   ├── services/
│   │   │   ├── notification/
│   │   │   │   ├── FirebasePushService.js
│   │   │   │   ├── NodemailerEmailService.js
│   │   │   │   └── TwilioSMSService.js
│   │   │   ├── event-bus/
│   │   │   │   └── EventEmitterBus.js
```

```
│   │   │   └── cache/
│   │   │       └── Redis no verificadoCacheService.js
│   │   └── config/
│   │       ├── database.js
│   │       ├── mqtt.js
│   │       ├── firebase.js
│   │       ├── redis.js
│   │       └── env.js
│   ├── events/
│   │   ├── handlers/
│   │   │   ├── ReadingCreatedHandler.js
│   │   │   ├── AlertTriggeredHandler.js
│   │   │   └── DeviceStatusChangedHandler.js
│   │   └── subscribers.js
│   ├── jobs/
│   │   ├── daily-summary.job.js
│   │   ├── monthly-summary.job.js
│   │   ├── generate-recommendations.job.js
│   │   ├── refresh-materialized-views.job.js
│   │   └── clean-audit-logs.job.js
│   ├── mqtt/
│   │   ├── mqtt.client.js
│   │   ├── mqtt.handlers.js
```

```
│   │   └── mqtt.topics.js
│   ├── shared/
│   │   ├── utils/
│   │   │   ├── logger.js
│   │   │   ├── jwt.js
│   │   │   ├── bcrypt.js
│   │   │   └── date-helper.js
│   │   ├── constants/
│   │   │   ├── roles.js
│   │   │   └── alert-types.js
│   │   └── exceptions/
│   │       └── AppError.js
│   └── app.js
├── tests/
│   ├── unit/
│   ├── integration/
│   ├── e2e/
│   └── fixtures/
├── .env
├── .gitignore
├── docker-compose.yml
├── package.json
└── server.js
```

Referencias

Servicio Nacional de Aprendizaje. (2025). *Especificación de requisitos de software – Hidro Smart* (Versión 1).

Motta Vargas, J. de J. (2025). *Acta general del proceso del proyecto Hidro Smart*.

Amaya Mosquera, J. D., Molano Cardozo, E. C., & Montealegre Rodríguez, J. A. (2026). *Encuesta sobre el consumo de agua*. Google Forms.

Anexo de actualización 2026: diseño frente a la implementación

El informe de diseño original se conserva por su amplitud, tablas, decisiones y diagramas. Este anexo identifica qué elementos corresponden al diseño vigente y cuáles deben considerarse objetivos o antecedentes.

Elemento de diseño	Estado verificable	Criterio de lectura
Arquitectura hexagonal / por capas	Respaldada por la organización del backend.	Se conserva como decisión arquitectónica.
Event-driven / MQTT	Respaldado en la cadena de telemetría.	Aplicable a IoT; no implica push de UI.
WebSockets / Socket.io	No encontrado como línea de ejecución vigente.	Tratar como evolución o requisito pendiente.
AWS/Firebase/Redis/Bull	No verificados en el corte.	Tratar como alternativas de diseño, no como dependencias instaladas.
TLS, backups diarios y cifrado en reposo	No comprobados end-to-end.	Mantener como controles de producción pendientes.
PostgreSQL/RLS/roles	Respaldados por BD_HS y documentación.	Nombres y permisos exactos se validan contra changelog.
BLE provisioning	Respaldado por firmware y frontend móvil.	Requiere development build y seguridad productiva adicional.

Componentes actuales que deben prevalecer

Mosquitto local, backend Express y PostgreSQL/Liquibase.

Mosquitto local, backend Express y PostgreSQL/Liquibase.

React/Vite web; experiencia móvil WebView con puente BLE.

React/Vite web; experiencia móvil WebView con puente BLE.

Topics hidrosmart/devices/{deviceCode}/telemetry, status, config y actuators.

Topics hidrosmart/devices/{deviceCode}/telemetry, status, config y actuators.

Persistencia y deduplicación de lecturas mediante mqttMessageId.

Persistencia y deduplicación de lecturas mediante mqttMessageId.

Polling HTTP de 5 segundos para reflejar datos persistidos en la UI.

Polling HTTP de 5 segundos para reflejar datos persistidos en la UI.